Measuring the Quality of Network Configurations Generated using LLM Models

Petr Matoušek, Michal Rozsíval, Aleš Smrčka, Jaromír Kotala
Brno University of Technology
Brno, Czech Republic
matousp, irozsival, smrcka@fit.vut.cz, xkotal01@stud.fit.vut.cz

***Abstract*—Large Language Models (LLMs) are a popular technology aiming to replace repetitive and tedious work. One such area is networking, where LLM models can assist human operators by generating device configurations based on natural language instructions. A crucial question is whether the generated configuration is both syntactically and semantically correct, and can therefore be deployed. This paper addresses the issue of how to evaluate the generated network configurations. Our framework uses a black box strategy, in which the LLM-based configuration system and the device configuration language remain hidden from the evaluation process. The evaluation comprises three dimensions: a static comparison of the generated configuration with the reference configuration, semantic validation of the task-specific behaviour, and regression validation of the previously established invariants. We measure the quality of the generated output using a multi-dimensional scoring system that includes a Configuration Accuracy Score (CAS), Weighted Feature Coverage (WFC), and Regression Preservation Rate (RPR). Our evaluation framework not only specifies whether the test has passed or failed, but also quantifies the number of mistakes generated. We demonstrate our approach using a set of non-trivial networking tasks involving Cisco IOS configurations generated by a selected LLM models.**

***Index Terms*—network configuration, evaluation, LLM, Cisco IOS, validation, functional tests, accuracy score**

I. Introduction

Computer networks are dynamic environments in which new devices and connections are added, new functionality is configured, security features are improved, and performance parameters are updated. All of these tasks require an experienced network engineer to update the existing configuration files. As device configurations can be considered a specific language, it seems straightforward to apply LLM models to generate them. Several studies describe how LLM models can assist with generating or translating network configurations [1]–[4]. The main question relating to the practical application and deployment of LLM-based configuration generation is how we can evaluate the accuracy, correctness, and reliability of systems that generate network configurations.

The accuracy of text generated by LLM models is typically measured using statistical metrics such as BLEU [5], ROUGE [6], Levenshtein distance, and others. These metrics calculate the precision of the match between an LLM-generated output and the reference output by either matching n-grams or by counting single-character edits such as insertions, deletions, and substitutions. However, such metrics are not very useful for comparing network configurations, where the order of keywords and the sequence of parameters are important, and even minor changes can result in a configuration that is faulty and unusable.

Supported by the project Novel AI-Driven Process Automation for Simplifying and Enhancing Telecommunication Processes (Spojit), 2024–2026, no. FW10010014, funded by the Technological Agency of the Czech Republic, and by the project NexTArc (no. 101194287) funded by the EU's Chips JU and the Czech national authority.

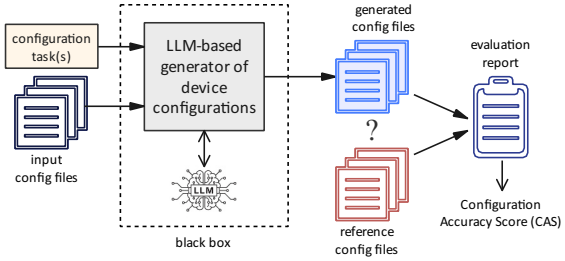


Fig. 1. Static comparison: black-box evaluation against a reference solution

The process of generating configuration files using an LLM is similar to the process of generating code, which mainly relies on unit testing. The unit testing process decomposes the code into logical parts called units, on which validation tests are run to prove the correctness of the generated code. Another approach is called property-based testing (PBT) [7], which verifies that the defined correctness properties are valid for a wide range of inputs. We apply both of these principles to our evaluation framework.

A big challenge in comparing the generated network configuration with the reference configuration is that the generated output may use different commands from the reference solution, yet both outputs could produce the same correct behaviour, i.e., semantic equivalence is preserved. Therefore, when comparing two configurations, we must also consider the semantics. The problem is how to check the semantics of a configuration without creating a full configuration language interpreter. One solution is to use existing emulators or verifiers, such as Batfish [8], which interpret a given configuration and verify that the behaviour of the generated configuration produces the same output as the reference configuration. Another approach is to apply pattern matching with static analysis and control-flow transformation [9].

To evaluate the generated network configuration, we pro-

2643-7929/26/$31.00 ©2026 IEEE
DOI 10.1109/NCA71379.2026.00013

pose a black box strategy with two approaches that hide the implementation of an LLM-based configuration system. The first approach, *static comparison*, evaluates the generated configuration against the reference configuration, see Figure 1. Since a network typically consists of several interconnected devices, the input to the LLM-based configuration system comprises one or more configuration files and an input task that specifies the required modification in plain language. The system then generates updated configuration files, which are then compared with the reference files.

However, the reference configuration is not always available. In such cases, the second approach, *test-based validation (semantic and regression)*, is applied. The tests can be created by a network expert or generated by an LLM in the form of validation functions, invariants, assertions, or functional block templates, see Figure 2. These tests are run in a controlled environment. In our case, we use the Pybatfish verifier[1] and the PNETLab emulator[2] to test Cisco IOS configurations.

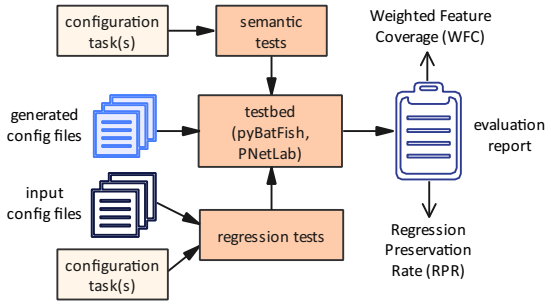


Fig. 2. Test-based validation of the generated configuration

The goal is to automate the evaluation process, although some parts of the evaluation will still require human assistance. Even if the generated configuration is only partially evaluated, valuable feedback on the quality of the result is still provided.

A. Contribution

The main contribution is an evaluation framework composed of static comparison, semantic validation, and regression validation. This framework is independent of the LLM-based system for network configuration generation and of the configuration language. The paper explains how to implement the framework and calculate the evaluation scores. A set of descriptive scores is defined using three quantitative metrics: the Configuration Accuracy Score (CAS) for static comparison, the Weighted Feature Coverage (WFC) for semantic validation, and the Regression Preservation Rate (RPR) for regression validation across consecutive tasks. These scores determine whether the generated configuration is correct in relation to the reference configuration and the defined tests. This is an essential consideration when the generated configuration is to be deployed on a real network. Additionally, we provide a set of typical scenarios with reference configurations for Cisco IOS devices, see our GitHub[3].

B. Paper Structure

Section II provides an overview of recent work on LLM-based network configuration systems, with a particular focus on evaluation. Section III presents the proposed evaluation framework and discusses the practical issues related to comparison and implementation. It also explains how to calculate the evaluation scores for a generated configuration. Section IV presents the application of our evaluation framework to a non-trivial real-world scenario. The final section concludes the study and discusses future research directions.

II. Related Work

Mondal et al. [1] apply LLM models to generate a simple configuration and translate it from Cisco to Juniper format. The authors combine the GPT-4 model with verifiers, using localised feedback from the verifiers to automatically correct errors. The configuration generation process runs in a loop until the generated output is verified as correct by manual inspection. For verification purposes, they employ a Batfish analysis tool [8], a Campion semantic verifier [10], and a topology verifier written in Python. To evaluate the results of configuration generation, they use a metric called leverage L, which is defined as the ratio of automated prompts to human prompts. Thus, the leverage L measures the potential improvement of the configuration generated by LLMs.

Chakraborty et al. [3] apply an LLM to generate a network configuration for an ISP network based on a tariff plan. The LLM analyses the tariff plan and the customer's specifications written by a human, identifying keywords and their values. These keyword-value pairs are then mapped to parameters in a predefined API template and handed over to the network orchestrator. The orchestrator then determines which network functions will be applied and invokes the APIs of the corresponding functions. Finally, a human operator checks the generated configuration. In their experiments, they used the Named Entity Recognition (NER) model, the fine-tuned GPT-3.5 Turbo model, and the fine-tuned Llama2-7B model.

Wei et al. [2] focus on translating network configurations using an LLM agent. They use an intent-based Retrieval-Augmented Generation (iRAG) system which splits a configuration file into fragments, extracts intents, and generates an accurate translation. The system also retrieves the configuration manual corresponding to the target device. The generated configuration is analysed using syntax and semantic verification. Syntax verification uses a command syntax tree and a configuration parser, while semantic verification analyses configuration differences using an LLM and semantic correction. The results are evaluated using the Tree Match metric, which is a measure of the matching rate on the configuration tree, the syntax matching rate, BLEU metrics, and the exact

[1]See https://pybatfish.readthedocs.io [Dec. 2025].
[2]See https://www.pnetlab.com [Dec. 2025].
[3]See https://github.com/MichalRozsival/configuration-agent/tree/main/scenarios [Sept. 2026].

match. Unfortunately, the paper does not explain how these metrics are computed.

In their paper, Kristina Dzeparoska et al. address intent-based networking [11]. As defined in RFC 9315 [12], intent-based networking is an approach in which the operator specifies his/her *intent* for network management using natural language. This intent is *a set of operational goals and outcomes that are defined in a declarative manner without specifying how to achieve or implement them* [12]. An example is "to maximise network utilisation of the link with respect to the SLA". The intent is then translated into individual configuration tasks and implemented on the relevant devices. Dzeparoska et al. utilise an LLM to decompose intents into policy-based abstractions, which are then mapped to APIs for intent execution and deployment. The authors employed few-shot learning to train an LLM model with an input comprising definitions (policy model), descriptions, and examples of intents, as well as the corresponding intent types and policy sequences. They employed the digital twin approach for validation, but did not provide details about the validation or evaluation of the correctness of the generated output.

The automated translation of natural language intentions into Network Service Descriptors (NSDs) is explored by Abdelkader Mekrache et al. [13]. Their work focuses on translating user intents for service configuration into NSDs. Their translation system incorporates a validation agent that checks syntax, semantics, and correlations. To improve accuracy, the authors created a knowledge base of previous intents. During translation, the system searches for the most similar intents based on cosine similarity.

Our evaluation framework is based on the work mentioned previously. As the overview shows, the various approaches use different methods to evaluate the generated configuration, making an objective comparison difficult. Therefore, we propose a black box approach that omits the generation process and focuses solely on the generated output. The evaluation scores describe how closely the generated configuration aligns with the reference configuration and the functional tests provided, as well as whether it is correct and can be deployed.

III. Evaluation Framework for LLM-based configuration systems

The aim of our evaluation framework is to decide whether the generated configuration satisfies the expected behaviour and to quantify any differences from the reference solution without taking into account how the configuration was generated. The evaluation consists of three dimensions: (i) a *static comparison* of configuration files, (ii) *semantic validation* of the task-specific behaviour, and (iii) *regression validation* of the previously established invariants. We formally introduce evaluation metrics for each of these dimensions to quantify the reliability and functionality of the generated configuration.

A. Formal framework

Definition 1. Let $E = (T, \prec, C, c_0, C^{ref}, \mathcal{V}, F, R)$ denote *an evaluation*, where $T = \{T_1, \ldots, T_n\}$ is a set of configuration tasks, $\prec$ is a precedence relation on T, C is a set of configurations, c_0 is the initial configuration, $C^{ref} = \{c_1, \ldots, c_n\}$ is a set of reference configurations, $\mathcal{V}$ is a set of validation tests, and $F = \{F_1, \ldots, F_n\}$ and $R = \{R_1, \ldots, R_n\}$ are collections of feature-test and regression-test suites, respectively, such that $F_i \subseteq \mathcal{V}$ and $R_i \subseteq \mathcal{V}$ for every $1 \leq i \leq n$.

Tasks T are written in natural language by a user familiar with networking, but not an expert in the specific configuration language. These tasks can be partially ordered by the precedence relation $\prec$, which captures the dependencies of the specific tasks. The set of configurations C contains the initial configuration c_0, the reference configurations produced by experts $C^{ref} = \{c_1, \ldots, c_n\}$, and the generated configurations C^{gen}. The reference configurations form a sequence that is constructed manually by applying each of the tasks in an admissible order with respect to $\prec$. This sequence contains exactly one reference configuration for each task and the notation c_i denotes the reference configuration corresponding to task T_i in this sequence. The reference sequence of configurations is $(c_1, c_2, \ldots, c_n)$, where c_i, $1 \leq i \leq n$, is obtained by a domain expert applying task T_i to c_{i-1}.

For each task T_i, we associate a feature-test suite $F_i \subseteq \mathcal{V}$ and a regression-test suite $R_i \subseteq \mathcal{V}$. Feature tests validate whether the configuration satisfies the requirements introduced by T_i. Regression tests validate whether the configuration preserves the invariants established by the already completed tasks and by the scenario as a whole. We denote by $GEN : C \times T \rightarrow C$ the *configuration-generation function* implemented by the LLM-based configuration system. Unlike the reference sequence of configurations, test-based validation is not limited to a single execution order: a generated configuration can be evaluated after any task sequence that respects the ordering relation $\prec$, and tests can be executed against any intermediate configuration in C^{gen}. For instance, the generated configuration $c_i^{gen} = GEN(c_{i-1}, T_i)$, $1 \leq i \leq n$, means that task T_i is sent to the LLM-based configuration system with the reference configuration c_{i-1} as the input configuration. This generated configuration can then be evaluated against both the reference configuration c_i and the tests in F_i and R_i. On the other hand, a configuration generated by $GEN(c, T_i)$, $c \neq c_{i-1}$, can only be validated by tests F_i and R_i since there is no reference configuration for it. We also denote by $passed(t, c) \in \{0, 1\}$ the result of test $t \in \mathcal{V}$ on configuration $c \in C$: the value 1 means that the test passed, and the value 0 means that it failed.

The evaluation dimensions are therefore defined as follows: the *static comparison* checks the similarity between c_i^{gen} and c_i, *semantic validation* checks whether F_i passes for the current task, and *regression validation* checks whether R_i remains satisfied after a new configuration has been generated.

B. Static comparison

Given the formal framework, static comparison compares the generated configuration $c_i^{gen} = GEN(c_{i-1}, T_i)$, $1 \leq i \leq n$, with the corresponding reference configuration c_i. Rather

than being limited to single-line comparison, as the `diff` command provides, it should instead cover *functional blocks*, which can represent an interface, a routing process, or an Access Control List (ACL). Cisco configurations, for example, use indentation to mark functional blocks, or functional blocks have the same prefix with the identifier, e.g., a list of numbered filtering rules.

The static comparison considers both syntactic differences and semantic equivalence. In addition, the nature of these differences in relation to real-world deployment is also taken into account. We define two classes of difference: *harmless* and *harmful*. *Harmless* differences may add or delete some configuration elements that do not affect the behaviour or functionality of the device. Examples include explicitly specifying the default configuration, implementing a more general solution than expected, or adding security-enhancing functionality that is not explicitly required by the user, see Figure 3. *Harmful* differences, on the other hand, negate the original configuration task, resulting in a non-functional configuration. These can be caused by commands that were incorrectly added or by commands that are missing in the generated configuration.

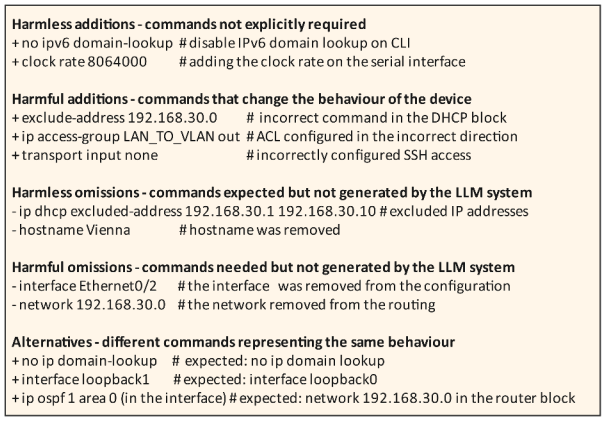


Fig. 3. The harmful and harmless differences in the configuration

The static comparison counts the following differences between a generated configuration c_i^{gen} and the reference configuration c_i:

- Added commands: the number of commands that were added to the generated configuration file compared to the reference file. We count the number of harmless additions a and the number of harmful additions a'.
- Missed commands: the number of commands that were not generated, but were expected by the reference configuration. We count the number of harmless omissions m and the number of harmful omissions m'.
- Alternatives: the number of commands that implement the requested behaviour using alternative commands. We count the number of harmless alternative commands t. No harmful alternatives are considered, since this category is covered by harmful additions or omissions.

To make the static comparison more comparable across tasks of different sizes, we introduce the *Configuration Accuracy Score* (CAS). The CAS metric for task T_i reflects the fact that any harmful change renders the configuration unsuitable for deployment. Therefore, if any harmful additions or omissions are present, i.e., if $a' > 0$ or $m' > 0$, we define $CAS_i = 0$. Otherwise, the CAS is defined as

$$CAS_i = 1 - \frac{w_a a + w_m m + w_t t}{N}, \tag{1}$$

where w_a, w_m, and w_t are non-negative weights reflecting the relative importance of different types of harmless change, and N denotes the number of lines in the generated configuration. For example, if there are five harmless changes in a generated file with 12 lines and the weights are set to 1, the CAS score will be 0.583. A higher score is achieved with fewer differences. A CAS score of 1 indicates that the generated and reference configuration files are syntactically identical[4]. Therefore, the CAS metric provides a single normalised value that can be used to make comparisons across tasks and models while explicitly penalising harmful changes.

Comparing alternatives in configuration files is challenging because two alternative configurations can be syntactically different, yet implement the same correct behaviour. Correct alternatives include the following subclasses:

- Functional equivalence: different commands implement the same functionality.
- Generalisation: the generated configuration is a superset of the reference solution, yet still satisfies the task.
- Specification: the generated configuration is a subset of the reference solution, yet still satisfies the task.

Figure 4 shows examples of three classes of alternatives that are not harmful.

Generated configuration	Reference configuration
Generalisation: access-list 100 permit icmp any any	access-list 100 permit icmp any 192.168.30.0 0.0.0.255 access-list 100 permit icmp any 192.168.40.0 0.0.0.255
Specification: router ospf 1 network 10.2.2.0 0.0.0.3 area 0 network 192.168.30.0 0.0.0.255 area 0 network 192.168.40.0 0.0.0.255 area 0	router ospf 1 network 0.0.0.0 255.255.255.255 area 0
Functional equivalence: ip route 0.0.0.0 0.0.0.0 192.168.20.1 router ospf 1 network 172.16.12.0 0.0.0.3 area 1 network 172.16.10.0 0.0.0.255 area 1 distribute-list 100 out Tunnel0 access-list 100 deny ip 172.16.10.0 0.0.0.255 any access-list 100 permit ip any any	ip route 0.0.0.0 0.0.0.0 Serial1/0 router ospf 1 passive-interface default no passive-interface Tunnel0 network 172.16.10.0 0.0.0.255 area 1 network 172.16.12.0 0.0.0.3 area 1

Fig. 4. Three classes of alternative configurations

A static comparison can compare the configuration trees of functional blocks [14] or structured configuration files [13]. We implemented a static comparison using a script that

[4]By setting a weight to $w = 0$, we can exclude unimportant changes from the comparison. In practice, we usually set $w_t = 0$, meaning that alternative differences are excluded from the final CAS score. In this case, $CAS = 1$, even if the files are not identical.

compares the functional blocks of the generated and reference configurations, identifies any changes, and counts the added and missed commands with respect to the reference configuration. Automatically identifying functional equivalence is not easy. One solution is to provide alternative configurations for selected functional blocks, such as routing, ACLs and DHCP, using templates.

The most important thing is to determine whether the change was harmful or harmless. To assist with this task, we use a database of frequent, harmless IOS configuration changes, which partially automates the process. However, the final decision should always be approved by a human expert.

C. Semantic validation

In common networking practice, updated configurations are verified through a series of task-specific tests. If these tests are successful, the configuration is deployed on a real network. If any test fails, the configuration is reviewed and updated. This approach is also essential in the case of configurations generated by LLM-based systems. However, a simple percentage of passed tests does not reflect that individual tests may differ in complexity and importance. Therefore, in addition to the typical pass/fail outcome, we define the *Weighted Feature Coverage* (WFC) for the feature tests associated with the current task.

In contrast to static comparisons, semantic validation focuses on the resulting behaviour of the generated configuration for the current task; therefore, it does not need to account for cases where the same configuration setting can be expressed in multiple forms. Moreover, semantic validation does not require a reference configuration to evaluate the quality and correctness of the generated network configuration, as tests can be written to assess the desired behaviour directly. Nevertheless, having a reference configuration can simplify the test design, as it enables both the reference configuration and the generated one to be subjected to the same tests. This form of differential testing enables a direct comparison of their respective responses, thereby facilitating the assessment of their functional equivalence.

Feature tests are task-specific assertions that verify whether the current configuration satisfies the requirements introduced by T_i. They can be designed by a network engineer or selected by an LLM from a database of validation functions for each configuration task. In our case, we implemented feature tests for the relevant network configuration categories, such as IP address assignment, NAT translation, OSPF routing and tunnelling. Feature tests that cover the common features can be reused for other scenarios that include configurations covered by these feature tests.

Feature tests can include:

- General router configuration tests, such as verifying the configured hostname, domain name and user accounts.
- Interface properties tests, including checks of the interface status, assigned IP addresses and applied OSPF settings.
- Access Control List (ACL) settings tests to verify their impact on network communication filtering.
- Reachability tests that check the availability of an end-to-end connection.

To quantify functional correctness, we define the *Weighted Feature Coverage* (WFC). Let F_i denote the set of *feature tests* associated with task T_i. Each test $t \in F_i$ is assigned a weight $w : F_i \to \mathbb{R}^+$. The weighted feature coverage for task T_i is then defined as

$$WFC_i = \frac{\sum_{t \in F_i} w(t)\, passed(t, c_i^{gen})}{\sum_{t \in F_i} w(t)}, \tag{2}$$

where $c_i^{gen} = GEN(c_{i-1}, T_i)$ is the generated configuration. If test suite F_i is empty, we define $WFC_i = 1$, which reflects the fact that there are no requirements to be satisfied for the given task.

This metric reflects the fact that some tests validate multiple configuration properties more thoroughly than single-assertion tests. In our experiments, the weights are assigned with respect to the impact of the verified property. As a practical approximation, the weight of a test is chosen proportional to the number of assertions it contains, which reflects the number of configuration properties validated by the test. Such approximation is consistent with the intuition that tests validating multiple properties provide stronger evidence of correctness.

D. Regression validation

Regression validation focuses on preserving the behaviour established in the preceding tasks. Unlike semantic validation, which evaluates whether the current task has been implemented correctly, regression validation ensures that the generated configuration does not unintentionally alter device functionality and that scenario-level invariants hold.

Let R_i denote the set of regression tests evaluated after task T_i, consisting of (i) general tests that verify basic configuration properties independently of T_i and (ii) all tests in F_j, $0 < j < i$, that validate behaviour for which task T_i does not request any modification. If the current task intentionally overrides behaviour established by a previous task, the affected tests are explicitly excluded from regression evaluation for the given task. Tests corresponding to future tasks are not included because the corresponding functionality has not yet been introduced. To quantify regression preservation, we define the *Regression Preservation Rate* (RPR) for task T_i as

$$RPR_i = \frac{|\{t \in R_i \mid passed(t, c_i^{gen}) = 1\}|}{|R_i|}, \tag{3}$$

where $c_i^{gen} = GEN(c_{i-1}, T_i)$ is the generated configuration. If test suite R_i is empty, we define $RPR_i = 1$, which reflects the fact that there are no invariants to be preserved for the given task.

This metric expresses the proportion of regression tests that remain satisfied after the configuration update for task T_i. A high RPR indicates that the generated configuration preserves the established invariants and does not introduce unintended disruptions to the previously validated behaviour.

TABLE I
EVALUATED LLM MODELS.

Model	Provider	Context window
GPT-5	OpenAI	400K
DeepSeek v3.2	DeepInfra (via OpenRouter)	163.8K
Grok 4.20	SpaceXAI (via OpenRouter)	2000K
GPT-OSS:20b	Local (via Ollama)	128K

IV. EXPERIMENTS

The proposed evaluation framework is demonstrated using a non-trivial network configuration involving three real Cisco 2900 series routers with Cisco IOS 15.7(3)M2, a Catalyst 2960 switch, and end stations with the Windows 11 OS, see Figure 5. The tasks are specified in natural language and sent to our LLM-based configuration agent [15], together with the default router configurations. The generated output is then compared with the reference configuration using static comparison, semantic and regression validations. The reference configuration was created using the PNETLab emulator[5].

The evaluation scores, as defined in Section III, are listed below. Full specifications of the individual tasks, the reference configurations, and the detailed outputs are available on GitHub[6].

A. Experimental Setup

Four large language models (LLMs), including the open-source GPT-OSS:20b, were employed for evaluation, see Table I. Each model was queried using the default decoding parameters; the sampling and generation settings, such as temperature, top_p, and max_tokens, were not explicitly configured. All models were evaluated using identical prompts and input conditions.

B. Network Scenario

Figure 5 shows the network scenario, which comprises two LANs connected via routers to an edge router and the internet. The configuration tasks include basic device configuration

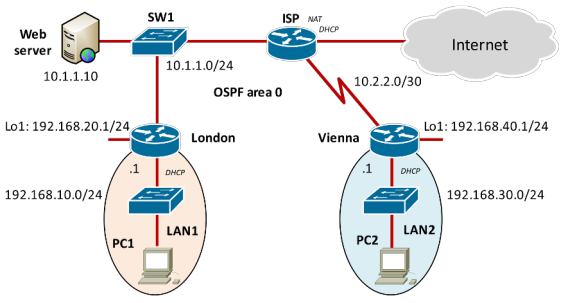


Fig. 5. Sample network scenario

(hostname, IP addresses, and admin account), DHCP server, SSH access, OSPF routing, and ACL filtering. These tasks are specified in natural language, see Figure 6, and sent to the LLM-based configuration agent, which then generates the requested configuration. This configuration is compared with the reference configuration (static comparison) and tested using semantic and regression tests. The final evaluation of the generated output is measured using the proposed scores: CAS, WFC, and RPR.

The configuration tasks are formulated by a user with networking experience, but who is not an expert in the specific configuration language. The LLM-based configuration agent then generates the requested configuration according to the provided task. We provide an initial configuration for each task, which is then updated, compared with the reference configuration, and tested. The first task begins with the default router configuration, which has no configured parameters. Then, this procedure is repeated using the same LLM-based configuration agent. The purpose of this empirical evaluation is to assess the quality of the generated configuration when using different LLM models.

```
Task 1
 - configure hostname Vienna, disable DNS lookup
 - create user admin with password cisco123 and root access

Task 2
 - configure serial interface s1/0 with IP address 10.2.2.2/30
 - configure LAN interface e0/0 with IP address 192.168.30.1/24
 - create a loopback interface with IP address 192.168.40.1/24

Task 3
 - configure  DHCP server for the LAN.
 - exclude the first 10 IP addresses from LAN 192.168.30.0/24
 - set default gateway to the 192.168.30.1
 - set DNS to 8.8.8.8

Task 4
 - use domain name vienna.com
 - allow only SSH access to the router

Task 5
 - configure OSPF routing, area 0
 - exchange routing information about all connected networks
 - do not send routing updates to local networks

Task 6
 - configure packet filtering for LAN at Vienna
 - LAN 192.168.30.0/24 is allowed to communicate with 192.168.40.0/24 (all traffic)
 - LAN 192.168.30.0/24 can access web server at the server 10.1.1.10
 - allow DNS and ICMP communication for all local networks.
 - no other traffic is allowed to originate from LAN 192.168.30.0/24
```

Fig. 6. Example of configuration tasks

The generation of the requested configuration was tested on four LLM models: GPT-5, DeepSeek, Grok, and GPT-OSS:20b, where we observed the quality of the generated output with respect to the reference configuration and expected behaviour using static comparison, semantic validation, and regression validation.

1) Static comparison: The results of the static comparison are shown in Tables II to V. The tables contain the number of added, missed, or alternative commands in two categories: harmless changes and harmful changes, see Section III. We also computed the overall Configuration Accuracy Score (CAS) defined by equation (1), which describes how close the generated output is to the expected configuration.

[5]See https://www.pnetlab.com/pages/main [June 2026].
[6]See https://github.com/MichalRozsival/configuration-agent/ [May 2026].

TABLE II
CONFIGURATION ACCURACY SCORE (CAS) FOR GPT-5

Task	a	m	t	a'	m'	$lines$	CAS
Task 1	1	0	1	0	0	71	92.9%
Task 2	0	0	2	0	0	71	100%
Task 3	0	1	1	0	0	77	87%
Task 4	0	0	1	4	1	89	0%
Task 5	0	0	1	0	0	83	100%
Task 6	12	0	0	6	2	106	0%

Recall that CAS is 0 if the generated configuration includes any harmful change.

As mentioned above, the CAS score reflects the difference between the generated configuration and the reference solution. In our network scenarios, we penalise missed commands in the generated configuration more heavily than added commands, so our weights are as follows: $w_a = 5$, $w_m = 10$, and $w_t = 0$. Alternative commands are not penalised at all ($w_t = 0$), because they are considered as equivalent correct solutions. Network administrators can adjust the weights to reflect the importance of changes.

TABLE III
CONFIGURATION ACCURACY SCORE (CAS) FOR DEEPSEEK

Task	a	m	t	a'	m'	$lines$	CAS
Task 1	0	1	0	0	1	70	0%
Task 2	0	0	2	0	0	71	100%
Task 3	0	0	1	0	0	77	100%
Task 4	1	1	0	0	1	80	0%
Task 5	2	0	0	0	0	85	88.2%
Task 6	0	0	0	17	0	100	0%

The value of CAS indicates how similar the generated solution is to the reference configuration. A non-zero value means that while the generated configuration is functional, it differs from the reference configuration. A zero CAS value means that the generated output contains serious errors that render the entire configuration faulty.

As can be seen from the tables, Tasks 1 to 3 were correctly generated by almost all of the tested LLM models, whereas Tasks 4 and 6 were not correctly generated by any of the models tested. A closer look at Tables II to V reveals that the GPT-OSS model produced a significant number of erroneous commands for Task 6 and removed 59 necessary commands when generating a configuration for Task 1, see Table V.

Surprisingly, none of the models correctly configured Task 4 (SSH configuration, see Figure 6). This is because natural language instructions can be ambiguous, causing the LLM model to interpret them incorrectly. Improving the task description affects the results and evaluation. For the complete generated outputs for all tasks, see our GitHub repository[3].

The presented CAS score can be used to evaluate the accuracy of network configurations generated by different LLM models, as well as to tune the parameters and prompts of the LLM model.

2) Semantic and regression validation: When designing the validation functions, we aim to make them as independent as possible so we can use them with other configurations beyond Cisco IOS. For this reason, the functions are primarily based on Pybatfish—a network analysis and validation tool that supports various vendors and languages. Furthermore, this tool also enables the complete verification of certain aspects of the configuration file. For example, it can be used to prove the reachability of specified network flows between given networks.

TABLE IV
CONFIGURATION ACCURACY SCORE (CAS) FOR GROK

Task	a	m	t	a'	m'	$lines$	CAS
Task 1	0	0	2	0	0	70	100%
Task 2	0	0	3	0	0	72	100%
Task 3	0	0	1	0	0	77	100%
Task 4	4	0	0	8	0	88	0%
Task 5	0	2	0	1	0	83	0%
Task 6	1	0	8	1	0	95	0%

TABLE V
CONFIGURATION ACCURACY SCORE (CAS) FOR GPT-OSS

Task	a	m	t	a'	m'	$lines$	CAS
Task 1	2	1	0	0	59	70	0%
Task 2	0	0	2	0	0	72	100%
Task 3	0	0	1	2	1	79	0%
Task 4	2	1	0	5	1	86	0%
Task 5	1	2	4	0	0	85	70.6%
Task 6	0	0	0	61	89	90	0%

As mentioned in Section III, in addition to the static comparison, which evaluates the differences between the generated and reference configurations, we also implement the semantic and regression validations. Semantic validation depends exclusively on the updated configuration files and the update task, see Figure 2. In this context, the validation functions act as specific assertions for the particular update task, ensuring that the updated configurations meet their objectives, see Figure 7.

```
Task 1: Hostname configuration:
  assert nodeProperties().Hostname == 'vienna'
Task 2: Interface Ethernet0/0 IP address configuration:
  assert interfaceProperties(interfaces='Ethernet0/0').PrimaryAddress ==
'192.168.30.1/24'
Task 4: domain name configuration:
  assert nodeProperties().Domain Name == 'vienna.com'
Task 5: OSPF area configuration:
  assert ospfProcessConfiguration().Areas == ['0']
Task 6: ACL filtering configuration:
  assert reachability(headers=(srcIps='192.168.30.0/24', dstIps='192.168.40.0/24'),
actions='Failure').empty
```

Fig. 7. Examples of pre-defined semantic tests

Regression validation requires updated configurations, an update task, and input configurations. Unlike semantic validation, the validation functions are used as a comparative mechanism and are assessed against both the input and output configurations. The results are then asserted to be identical. Using a predefined set of comparators ensures consistency across various aspects between the input and output configurations. Consequently, this validation approach only needs

to specify which comparator to exclude for each task. Specifically, the excluded comparators are those that evaluate aspects influenced by the update task. This per-task-derived subset of comparators can be used to verify that the LLM model has modified only the intended parts of the configurations while maintaining the integrity of the unaffected sections.

The results of the semantic and regression validations are summarised in Table VI. In our experiments, the WFC and RPR evaluations use constant weights set to 1.

TABLE VI
RESULTS OF THE SEMANTIC AND REGRESSION VALIDATIONS

Task	GPT-5		DeepSeek		Grok		GPT-OSS	
	WFC	*RPR*	*WFC*	*RPR*	*WFC*	*RPR*	*WFC*	*RPR*
Task 1	3/3	5/5	1/3	5/5	3/3	5/5	3/3	4/5
Task 2	3/3	5/5	3/3	5/5	3/3	5/5	3/3	5/5
Task 3	2/3	9/9	3/3	9/9	3/3	9/9	2/3	9/9
Task 4	2/3	8/8	3/3	7/8	2/3	8/8	2/3	7/8
Task 5	2/2	5/5	2/2	5/5	0/2	5/5	1/2	5/5
Task 6	0/4	9/9	2/4	9/9	4/4	9/9	0/4	5/9

The table outlines the success rates for each task and model, including the number of tests passed and the total number of tests conducted. In addition, we computed the Weighted Feature Coverage (WFC), where the weight of each test is proportional to the number of assertions it contains, and the Regression Preservation Rate (RPR), which evaluates whether the generated configuration preserves the expected progression of scenarios across consecutive tasks. The corresponding quantitative results are in Table VII.

C. Evaluation of the complex SNMP scenario

The complex SNMP scenario in Figure 8 demonstrates the viability and advantages of the proposed multi-dimensional evaluation framework. This scenario includes seven routers and two firewalls. Firewall FW1 filters all incoming traffic except traffic from the SNMP server, while FW2 only allows web, email and SSH services to the router agents[7].

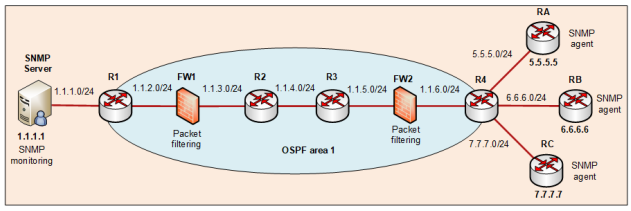


Fig. 8. SNMP scenario

In the SNMP scenario, an SNMP configuration including SNMP polling and SNMP traps is requested on routers RA, RB, and RC. Firewall updates are also required on intermediate devices to enable SNMP communication between the SNMP server and routers RA, RB and RC.

[7]See the details and input configurations on our GitHub at https://github.com/MichalRozsival/configuration-agent/tree/main/scenarios/snmp [Sept. 2026]

Therefore, the LLM-based configuration agent is expected to generate the necessary changes to all the relevant devices. This involves selecting the correct devices to be updated, identifying the relevant configuration blocks to be updated and making the appropriate additions or deletions to the existing configuration. The final result is then evaluated using both static (CAS metric) and dynamic (WFC and RPR metrics) comparisons.

Since this complex task involves updating more devices, the proposed metrics are straightforwardly extended to cover more devices. This means that the evaluation parameters, i.e., the number of additions/deletions for CAS, the number of functional tests passed for WFC, and the number of passed invariants for RPR, are summarised across multiple devices. The total score therefore covers the entire updated network configuration. In the case of the SNMP scenario, the total metrics include a static comparison of nine devices and dynamic tests applied to the entire topology.

The evaluation of the generated configurations can be found in Table VIII. It summarises the CAS, WFC and RPR metrics for all configurations generated by each LLM model. For GPT-OSS, the same task was run repeatedly to see how much the generated results differed.

The table clearly illustrates the advantages of a multi-dimensional evaluation approach. While almost all generated configurations correctly preserve the behaviour expressed by the RPR tests, most of them fail the functional tests expressed by the WFC metric. They also differ significantly from the reference configuration (ground truth), as indicated by the CAS metric.

If a failure is detected during WFC or RPR tests, the user can repeatedly re-run the network configuration generation process, providing additional data about the failure and requesting that it be avoided. This can improve the generation process. However, this should be done under human control to prevent infinite looping.

D. Discussion

Our experiments demonstrate how our proposed evaluation framework can be used to objectively assess network configurations produced by LLM models. As many research teams aim to improve the generation of network configurations using LLMs, we believe that the framework will help them to evaluate their results in comparison to those of others.

The introduced quantitative metrics provide a more fine-grained view of the quality of the generated outputs. CAS normalises the static differences with respect to the size of the reference configuration, WFC measures the success rate of functional tests for a given task, and RPR reflects whether the generated configuration evolves consistently across consecutive tasks. This helps to reveal cases where a model breaks previously established functionality. Together, these three metrics are important for identifying potential errors in generated configurations.

When we compare our framework with existing approaches and evaluation metrics such as BLEU, ROUGE, Levenshtein

TABLE VII
EVALUATION USING CONFIGURATION ACCURACY SCORE (CAS), WEIGHTED FEATURE COVERAGE (WFC), REGRESSION PRESERVATION RATE (RPR)

Task	GPT-5			DeepSeek			Grok			GPT-OSS		
	CAS	*WFC*	*RPR*	*CAS*	*WFC*	*RPR*	*CAS*	*WFC*	*RPR*	*CAS*	*WFC*	*RPR*
Task 1	92.9%	100%	100%	0%	33.33%	100%	100%	100%	100%	0%	100%	80%
Task 2	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
Task 3	87%	66.67%	100%	100%	100%	100%	100%	100%	100%	0%	66.67%	100%
Task 4	0%	66.67%	100%	0%	100%	87.5%	0%	66.67%	100%	0%	66.67%	87.5%
Task 5	100%	100%	100%	88.2%	100%	100%	0%	0%	100%	70.6%	50%	100%
Task 6	0%	0%	100%	0%	50%	100%	0%	100%	100%	0%	0%	55.56%

TABLE VIII
EVALUATION OF THE SNMP SCENARIO. (*) CAS COMPUTED WITHOUT CLASSIFYING CHANGES AS HARMLESS OR HARMFUL.

Model	CAS (*)	WFC	RPR
GPT-5	79.6%	100%	100%
DeepSeek	74.5%	58.3%	100%
Grok	89.4%	91.7%	100%
GPT-OSS (run 1)	52.0%	58.3%	100%
GPT-OSS (run 2)	68.5%	41.7%	100%
GPT-OSS (run 3)	61.4%	100%	100%
GPT-OSS (run 4)	64.7%	50%	100%
GPT-OSS (run 5)	83.2%	66.7%	97.8%

distance and cosine similarity, we find that our proposed approach can measure not only the syntactic differences between a generated solution and a reference configuration, but also the semantics of generated configurations as expressed by functional and regression tests. This is an especially important consideration for practical deployment, where proper functionality is critical.

The results presented in Table VII indicate that none of the evaluated models achieves safe configurations across all tasks. In particular, the CAS reaches 0% at least once for each model, indicating configurations that deviate from the reference solution to the extent that compromises safe operation. This is further supported by the fact that at least one test fails in each of the evaluated test suites, showing that even a single misconfiguration can invalidate the intended behaviour, despite high aggregate scores elsewhere.

Comparison of metrics further shows that CAS is more sensitive to such deficiencies than semantic and regression validation. While both WFC and RPR often report high scores, suggesting that the generated configurations satisfy the functional requirements and preserve the expected behaviour, the CAS score can be zero due to a single harmful change. In Task 6 for Grok, a static comparison reveals harmful changes ($CAS = 0$) that are not detected by the test suites ($WFC \cdot RPR = 1$), indicating that the test suites do not cover all the relevant configuration aspects. On the other hand, in Task 3 for GPT-5 and Task 5 for GPT-OSS, the generated configuration contains harmless generalisation ($CAS > 0$), which is not reflected in the test results ($WFC \cdot RPR < 1$). This suggests that the test suite strictly validates the intended behaviour, without allowing for alternative implementations.

Overall observations suggest that CAS is more sensitive than WFC and RPR to structural deviations from the reference configuration. In many cases, it provides a more reliable warning about potentially unsafe outputs. This sensitivity is partly due to the manual expert review involved in classifying configuration differences, which can identify harmful changes that are not covered by the current tests. However, manual involvement is also a disadvantage of CAS, as it requires an expert-level knowledge of both the configuration language and the intended network semantics. Therefore, extending the test suite is a more practical solution than repeatedly classifying configuration differences manually. Although designing tests still requires an understanding of the intended network semantics, maintaining the encoded suite is easier, particularly when supported by queries over general network flows.

The evaluation results suggest that improving LLM-based configuration systems requires more than just prompt-level adjustments. The discrepancy between high WFC/RPR scores and low CAS scores indicates that models can fulfil individual test cases while deviating from the intended global configuration semantics. This highlights the need to integrate verification feedback into the generation process. For example, configurations could be refined based on issues identified by analysis tools such as Batfish, or higher-level constraints could be enforced over network-wide properties. Overall, the results confirm that verification and testing are insufficient when used only for final validation, and must be incorporated as active components of the configuration generation workflow.

V. CONCLUSION

The proposed evaluation framework is part of our research project, in which we are developing an LLM-based configuration agent for a global telecommunications company. While experimenting with different prompts, Retrieval-Augmented Generation (RAG), LLM models and their parameters using the same set of configuration scenarios for testing purposes, we realised the need for a methodology to assess the correctness of the configurations generated. Unable to find a suitable existing methodology, we defined a new framework that works well for our purposes. The framework is designed in such a way that it can be applied to any form of network configuration. For example, when generating Juniper configurations, only minor adjustments are required to implement the proposed

static comparison, semantic validation and regression validation processes.

The experimental results demonstrate the feasibility of the proposed evaluation framework. A combination of static comparison, semantic validation and regression validation is important because some configuration errors can be easily detected by static comparison, while others can only be revealed by test-based validation. We have developed a tool that supports the semi-automated evaluation of pre-defined configuration tasks.

In addition to the above scenario, we have also prepared and evaluated various scenarios covering IPv6 networking and configuration, VPN configuration using GRE/IPsec, MPLS scenarios, and HSRP configuration. The complete set of scenarios, task assignments, reference configurations, and functional tests is available on our GitHub[3].

REFERENCES

[1] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?" in *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, ser. HotNets '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 189–195.

[2] Y. Wei, X. Xie, Y. Zuo, T. Hu, X. Chen, K. Chi, and Y. Cui, "Leveraging LLM Agents for Translating Network Configurations," 2025.

[3] S. Chakraborty, N. Chitta, and R. Sundaresan, "Automation of Network Configuration Generation using Large Language Models," in *2024 20th International Conference on Network and Service Management (CNSM)*, 2024, pp. 1–7.

[4] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A General Approach to Network Configuration Verification," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 155–168.

[5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ser. ACL '02. USA: Association for Computational Linguistics, 2002, p. 311–318.

[6] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Annual Meeting of the Association for Computational Linguistics*, 2004. [Online]. Available: https://api.semanticscholar.org/CorpusID:964287

[7] Z. Paraskevopoulou, C. HriŢcu, M. Dénès, L. Lampropoulos, and B. C. Pierce, "Foundational property-based testing," in *Interactive Theorem Proving*, C. Urban and X. Zhang, Eds. Cham: Springer International Publishing, 2015, pp. 325–343.

[8] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'15. USA: USENIX Association, 2015, p. 469–483.

[9] V. Malík and T. Vojnar, "Automatically Checking Semantic Equivalence between Versions of Large-Scale C Projects," in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021, pp. 329–339.

[10] A. Tang, S. K. R. Kakarla, R. Beckett, E. Zhai, M. Brown, T. Millstein, Y. Tamir, and G. Varghese, "Campion: debugging router configuration differences," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, ser. SIGCOMM '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 748–761.

[11] K. Dzeparoska, J. Lin, A. Tizghadam, and A. Leon-Garcia, "LLM-Based Policy Generation for Intent-Based Management of Applications," in *2023 19th International Conference on Network and Service Management (CNSM)*, 2023, pp. 1–7.

[12] A. Clemm, L. Ciavaglia, L.Z.Granville, and J. Tantsuare, "*Intent-Based Networking—Concepts and Definitions*," IETF, RFC 9315, October 2022.

[13] A. Mekrache and A. Ksentini, "LLM-enabled intent-driven service configuration for next generation networks," in *2024 IEEE 10th International Conference on Network Softwarization (NetSoft)*. IEEE, 2024, pp. 253–257.

[14] K. Nakamura, H. Suzuki, S. Ogata, H. Hashiura, T. Nagai, and K. Okano, "A Method to Automatically Extract a Network Device Configuration Model by Parsing Network Device Configurations," 2025. [Online]. Available: https://arxiv.org/abs/2511.17948

[15] M. Rozsíval, P. Matoušek, and J. Kotala, "Poster: Multi-Agent LLM System for Cisco Router Configuration," in *2025 23rd International Symposium on Network Computing and Applications (NCA)*, 2025, pp. 306–307.